CM2005 Object-Oriented Programming
End-term
Report: Coursework 2

TABLE OF CONTENTS

1. INTRODUCTION **2**
2. BASIC FUNCTIONALITIES **2**
1.1. Load Audio Files 2
1.1.1. Load from DeckGUI 2
1.1.2. Load from PlaylistComponent 3
1.2. Play Two Tracks 6
1.3. Mix Tracks By Varying Each of Their Volumes 8
1.4. Speed Up and Slow Down 9
3. USER INTERFACE CUSTOMISATION **12**
2.1. Dual Deck Symmetry Control GUI 12
2.2. Text in UI 15
2.2.1. Slider Labels 16
2.2.2. Deck Identification 16
2.3. Drawable `playPauseButton` 17
2.3.1. Loading Play and Pause Icons 17
2.3.2. Setting Button Images 18
2.3.3. Color Customization 18
2.3.4. Toggling Play and Pause 18
2.4. PlaylistComponent 19
4. NEW FEATURE **21**
3.1. Research 21
3.2. Feature Identified to Implement 22
3.2.1. Transition Slider 22
3.2.2. Movable Playhead 23
5. CONCLUSION **25**
6. REFERENCES **25**

1. INTRODUCTION

This report outlines the further development of Otodecks, a basic DJ application, by enhancing its functionality and user experience. The report begins with a description of how the original program operates, detailing its core features and structure. It then explores the process of customizing the user interface, highlighting the design choices made to improve usability and visual appeal. Finally, the report presents research and technical insights into identifying, analyzing, and implementing a new feature inspired by other DJ applications, demonstrating how these improvements elevate Otodecks beyond its initial capabilities.

2. BASIC FUNCTIONALITIES

1.1. Load Audio Files

1.1.1. Load from DeckGUI

In the DeckGUI component, audio files are loaded using a file chooser dialog. When the user clicks the **`LOAD` button**, the application initiates an asynchronous file selection process using `launchAsync()`.

```
private:
    TextButton loadButton{ "LOAD" };
```

```
// void DeckGUI::buttonClicked(Button * button)
if (button == &loadButton)
{
    auto fileChooserFlags = FileBrowserComponent::canSelectFiles;
    fChooser.launchAsync(fileChooserFlags, [this](const FileChooser& chooser)
        {
            File chosenFile = chooser.getResult();
            player->loadURL(URL{ chosenFile });
            waveformDisplay.loadURL(URL{ chosenFile });
        });
}
```

This allows the user to browse and select an audio file from their system. Once a file is chosen, it is passed as a `File` object to the function handling the selection. The `URL` of the selected file is then used to load the track into the audio player (**`player->loadURL(URL{ chosenFile })`**)

```
void DJAudioPlayer::loadURL(URL audioURL)
{
    auto* reader = formatManager.createReaderFor(audioURL.createInputStream(false));
    if (reader != nullptr) // good file!
    {
        std::unique_ptr<AudioFormatReaderSource> newSource(new AudioFormatReaderSource(reader, true));
        transportSource.setSource(newSource.get(), 0, nullptr, reader->sampleRate);
        readerSource.reset(newSource.release());
    }
}
```

and simultaneously update the waveform display (`**waveformDisplay.loadURL(URL{ chosenFile })**`). This ensures that the user can see a visual representation of the audio waveform before playback begins.

1.1.2. Load from PlaylistComponent

On the other hand, the PlaylistComponent provides another method for loading audio tracks, specifically from a pre-existing list of songs stored in the application. The PlaylistComponent first loads **all '.mp3' files** with function `**loadTracksFromDirectory()**` and stores them into the corresponding **vectors**.

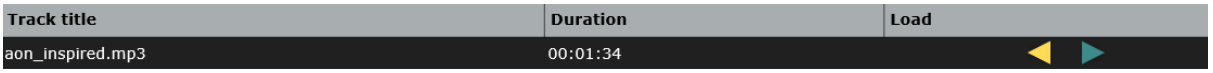
```
// private:
    /* Vectors to store tracks */
    std::vector<std::string> trackTitles;
    std::vector<std::string> trackDurations;
    std::vector<URL> trackURLs;

    /* Load tracks' titles, durations, URLs from "Tracks" directory */
    void loadTracksFromDirectory(const File& directory);
```

```
void PlaylistComponent::loadTracksFromDirectory(const File& directory)
{
    if (directory.exists() && directory.isDirectory())
    {
        // Set up the AudioFormatManager to recognize MP3 files
        AudioFormatManager formatManager;
        formatManager.registerBasicFormats();

        // Find all .mp3 files in the directory
        Array<juce::File> mp3Files;
        directory.findChildFiles(mp3Files, juce::File::findFiles, false, "*.mp3");

        for (const auto& file : mp3Files)
        {
            // Add their names to `trackTitles` vector
            trackTitles.push_back(file.getFileName().toStdString());

            // Add their durations to `trackDurations` vector
            std::unique_ptr<juce::AudioFormatReader> reader(formatManager.createReaderFor(file));
            if (reader)
            {
                double durationInSeconds = reader->lengthInSamples / reader->sampleRate;
                std::string formattedDuration = formatDuration(durationInSeconds); // Format as "HH:MM:SS"
                trackDurations.push_back(formattedDuration);
            }

            // Add their URLs to `trackURLs` vector
            trackURLs.push_back(URL{ file });
        }
        tableComponent.updateContent(); // Refresh the table
    }
    else
    {
        DBG("Directory does not exist or is not a directory: " + directory.getFullPathName());
    }
}
```

Then generates **two separate load buttons** under the "Load" column for each track in the playlist. This is handled in the `refreshComponentForCell` function, which ensures that each row in the playlist table contains two arrow buttons—one for *deckGUI1 and another for *deckGUI2.

```
Component* PlaylistComponent::refreshComponentForCell(int rowNumber,
                                                      int columnId,
                                                      bool isRowSelected,
                                                      Component* existingComponentToUpdate)
{
    if (columnId == 3) // Load column
    {
        if (existingComponentToUpdate == nullptr)
        {
            auto* buttonContainer = new Component();
            buttonContainer->setSize(250, 30);

            auto* loadToDeck1 = new ArrowButton ("Load to Deck 1", 0.5f, Colour::fromRGB(255, 219, 88));
            loadToDeck1->setSize(20, 20);
            loadToDeck1->setTopLeftPosition(250/2-20, 2);
            loadToDeck1->addListener(this);
            loadToDeck1->setComponentID("deck1_" + String(rowNumber)); // Store row number in ID
            buttonContainer->addAndMakeVisible(loadToDeck1);

            auto* loadToDeck2 = new ArrowButton("Load to Deck 2", 0.0f, Colour::fromRGB(62, 137, 137));
            loadToDeck2->setSize(20, 20);
            loadToDeck2->setTopLeftPosition(250/2+20, 2);
            loadToDeck2->addListener(this);
            loadToDeck2->setComponentID("deck2_" + String(rowNumber)); // Store row number in ID
            buttonContainer->addAndMakeVisible(loadToDeck2);

            existingComponentToUpdate = buttonContainer;
        }
    }

    return existingComponentToUpdate;
}
```

These buttons are visually distinct, with `"Load to Deck 1"` styled in a yellowish color (`Colour::fromRGB(255, 219, 88)`) and `"Load to Deck 2"` styled in a teal color (`Colour::fromRGB(62, 137, 137)`). Each button is assigned a unique **`component ID`** that embeds the row number, allowing the system to correctly identify and load the selected track into the appropriate deck when clicked.



After that, the function **`buttonClicked(Button* button)`** will be triggered when a user clicks a button associated with a track in the playlist. The function extracts the button's unique ID, which encodes the track's row number in the playlist and the intended deck (Deck 1 or Deck 2). If the button's ID starts with `"deck1_"`, the system retrieves the corresponding track's URL using **`getTrackURL(rowNumber)`** and loads it into Deck 1's player and waveform display. Similarly, if the button's ID starts with `"deck2_"`, the track is loaded into Deck 2's player and waveform display.

```
URL PlaylistComponent::getTrackURL(int rowNumber)
{
    if (rowNumber >= 0 && rowNumber < trackURLs.size())
        return trackURLs[rowNumber];
    return URL();
}
```

```
void PlaylistComponent::buttonClicked(Button* button)
{
    // Extract the row number and deck type from the button's ID
    String buttonID = button->getComponentID();
    int rowNumber = buttonID.getTrailingIntValue();

    if (buttonID.startsWith("deck1_")) // Load to Deck 1
    {
        URL trackURL = getTrackURL(rowNumber);
        player1->loadURL(trackURL);
        waveformDisplay1->loadURL(trackURL);
    }
    else if (buttonID.startsWith("deck2_")) // Load to Deck 2
    {
        URL trackURL = getTrackURL(rowNumber);
        player2->loadURL(trackURL);
        waveformDisplay2->loadURL(trackURL);
    }
}
```

This approach ensures that users can quickly send a track from the playlist to their preferred deck without manually selecting a file each time.

1.2. Play Two Tracks

Playing two tracks simultaneously is achieved by utilizing two instances of the `**DJAudioPlayer**` class and a `**MixerAudioSource**` to combine their outputs. The tracks are represented by `**player1**` and `**player2**`, each managing a separate audio source.

The `**MainComponent**` class holds references to both `DJAudioPlayer` instances (`player1` and `player2`) as well as the `MixerAudioSource` (`mixerSource`):

```
private:
    //==============================================================================
    // Your private member variables go here..
    AudioFormatManager formatManager;
    AudioThumbnailCache thumbCache{ 100 }; // store up to 100 waveforms

    DJAudioPlayer player1{ formatManager };
    DJAudioPlayer player2{ formatManager };
    MixerAudioSource mixerSource;
```

During initialization, the `**prepareToPlay()**` method is called for both `DJAudioPlayer` instances and the mixer, ensuring that the audio sources are prepared to play at the specified sample rate and block size. Afterward, the `**addInputSource()**` method is used to add both `**player1**` and `**player2**` to the mixer. This step allows the mixer to handle and combine the audio from both players, making it possible to play both tracks together.

```
void MainComponent::prepareToPlay(int samplesPerBlockExpected, double sampleRate)
{
    player1.prepareToPlay(samplesPerBlockExpected, sampleRate);
    player2.prepareToPlay(samplesPerBlockExpected, sampleRate);
    mixerSource.prepareToPlay(samplesPerBlockExpected, sampleRate);

    mixerSource.addInputSource(&player1, false);
    mixerSource.addInputSource(&player2, false);
}
```

In the `**getNextAudioBlock()**` method, the mixer is instructed to fill the audio buffer with the combined audio from both tracks. The mixer takes care of routing the audio from each player to the output.

```
void MainComponent::getNextAudioBlock(const AudioSourceChannelInfo& bufferToFill)
{
    mixerSource.getNextAudioBlock(bufferToFill);
}
```

Finally, the `**releaseResources()**` method ensures that the resources used by both `DJAudioPlayer` instances and the mixer are properly released when playback stops or when the audio device is restarted.

```
void MainComponent::releaseResources()
{
    // This will be called when the audio device stops, or when it is being
    // restarted due to a setting change.
    player1.releaseResources();
    player2.releaseResources();
    mixerSource.releaseResources();
}
```

In the `**DJAudioPlayer**` class, each player has a `**transportSource**`, which handles the playback of the audio file. The `prepareToPlay()` and `releaseResources()` methods in `DJAudioPlayer` ensure that the transport source is initialized and cleaned up appropriately for each track.

```
AudioTransportSource transportSource;
```

```
void DJAudioPlayer::prepareToPlay(int samplesPerBlockExpected, double sampleRate)
{
    transportSource.prepareToPlay(samplesPerBlockExpected, sampleRate);
    resampleSource.prepareToPlay(samplesPerBlockExpected, sampleRate);
}
```

```
void DJAudioPlayer::releaseResources()
{
    transportSource.releaseResources();
    resampleSource.releaseResources();
}
```

This approach allows for the simultaneous playback of two tracks, managed and mixed together by the `**MixerAudioSource**`, enabling a seamless DJ experience.

1.3. Mix Tracks By Varying Each of Their Volumes

By `**setGain()**` and `**getGain()**` methods of the `**DJAudioPlayer**` class, we can control the volume of each track independently.

The `**setGain()**` method in `DJAudioPlayer` allows adjusting the gain (volume) of the track within a range of 0.0 to 2.0, where 0.0 mutes the track and 2.0 doubles its volume. If an invalid value is provided (outside this range), a debug message is logged, ensuring that only valid gain values are applied.

The `**getGain()**` method retrieves the current gain of the track, which can be useful for displaying the current volume or making further adjustments.

```
void DJAudioPlayer::setGain(double gain)
{
    if (gain < 0 || gain > 2.0)
    {
        DBG("DJAudioPlayer::setGain gain should be between 0 and 2.");
    }
    else
    {
        transportSource.setGain(gain);
    }
}

double DJAudioPlayer::getGain()
{
    return transportSource.getGain();
}
```

To allow the user to control the volume of each track, a `**Slider**` component (`**volSlider**`) is used in the `**DeckGUI**` class. This slider is set to range from 0.0 to 2.0, with an initial value of 1.0 (neutral volume). The slider's value is displayed with one decimal place, and its snapping behavior is disabled for more precise control.

```
Slider volSlider{ Slider::LinearVertical, Slider::TextBoxBelow };
```

```
    /* Adding Listeners */
    playPauseButton.addListener(this);
    loadButton.addListener(this);
    volSlider.addListener(this);
    speedSlider.addListener(this);

//== Sliders configurations ==================================================
    /* Set range of sliders */
    volSlider.setRange(0.0, 2.0);
    speedSlider.setRange(0.1, 3.0);

    /* Set default values for sliders */
    volSlider.setValue(1.0);
    speedSlider.setValue(1.0);

    /* Limit slider values to increments of 0.1, display only 1 decimal place */
    volSlider.setNumDecimalPlacesToDisplay(1);
    speedSlider.setNumDecimalPlacesToDisplay(1);

    /* Disable snapping to mouse position */
    volSlider.setSliderSnapsToMousePosition(false);
```

Whenever the slider value changes, the `**sliderValueChanged()**` method is triggered. Inside this method, if the slider in question is `volSlider`, the value is rounded to one decimal place for smoother adjustments. The rounded value is then passed to the `setGain()` method of the corresponding `DJAudioPlayer` (`player`), updating the track's volume in real time.

```
void DeckGUI::sliderValueChanged(Slider* slider)
{
    if (slider == &volSlider)
    {
        double roundedValue = round(volSlider.getValue() * 10) / 10; // Round to 1 decimal place
        player->setGain(roundedValue);
    }
```

This setup allows users to control the volume of each track independently, enabling effective mixing by adjusting the gain of the individual audio sources.

1.4. Speed Up and Slow Down

Adjusting the speed of playback is achieved through the `**setSpeed()**` method in the `**DJAudioPlayer**` class, which uses a `**ResamplingAudioSource**` to modify the playback speed of the audio.

```
ResamplingAudioSource resampleSource{ &transportSource, false, 2 };
```

The `**ResamplingAudioSource**` is a component that allows for the resampling of the audio, effectively altering its playback speed.

The `**setSpeed()**` method accepts a ratio that determines how much faster or slower the audio will play. The ratio is constrained between 0.1 and 3.0, with 1.0 representing normal speed. If the ratio is outside this range, a debug message is logged to ensure only valid values are used. When the `setSpeed()` method is called with a valid ratio, it updates the `resampleSource`'s `setResamplingRatio()` method, which adjusts the playback speed accordingly.

```
void DJAudioPlayer::setSpeed(double ratio)
{
    if (ratio < 0.1 || ratio > 3.0)
    {
        DBG("DJAudioPlayer::setSpeed ratio should be between 0.1 and 3.");
    }
    else
    {
        resampleSource.setResamplingRatio(ratio);
    }
}
```

The `**prepareToPlay()**` and `**getNextAudioBlock()**` methods in `DJAudioPlayer` ensure that the `resampleSource` is prepared for playback and that the resampled audio data is passed through to the output during playback. The `**releaseResources()**` method ensures that the `resampleSource`'s resources are properly cleaned up when playback ends.

```
void DJAudioPlayer::prepareToPlay(int samplesPerBlockExpected, double sampleRate)
{
    transportSource.prepareToPlay(samplesPerBlockExpected, sampleRate);
    resampleSource.prepareToPlay(samplesPerBlockExpected, sampleRate);
}

void DJAudioPlayer::getNextAudioBlock(const AudioSourceChannelInfo& bufferToFill)
{
    resampleSource.getNextAudioBlock(bufferToFill);
}

void DJAudioPlayer::releaseResources()
{
    transportSource.releaseResources();
    resampleSource.releaseResources();
}
```

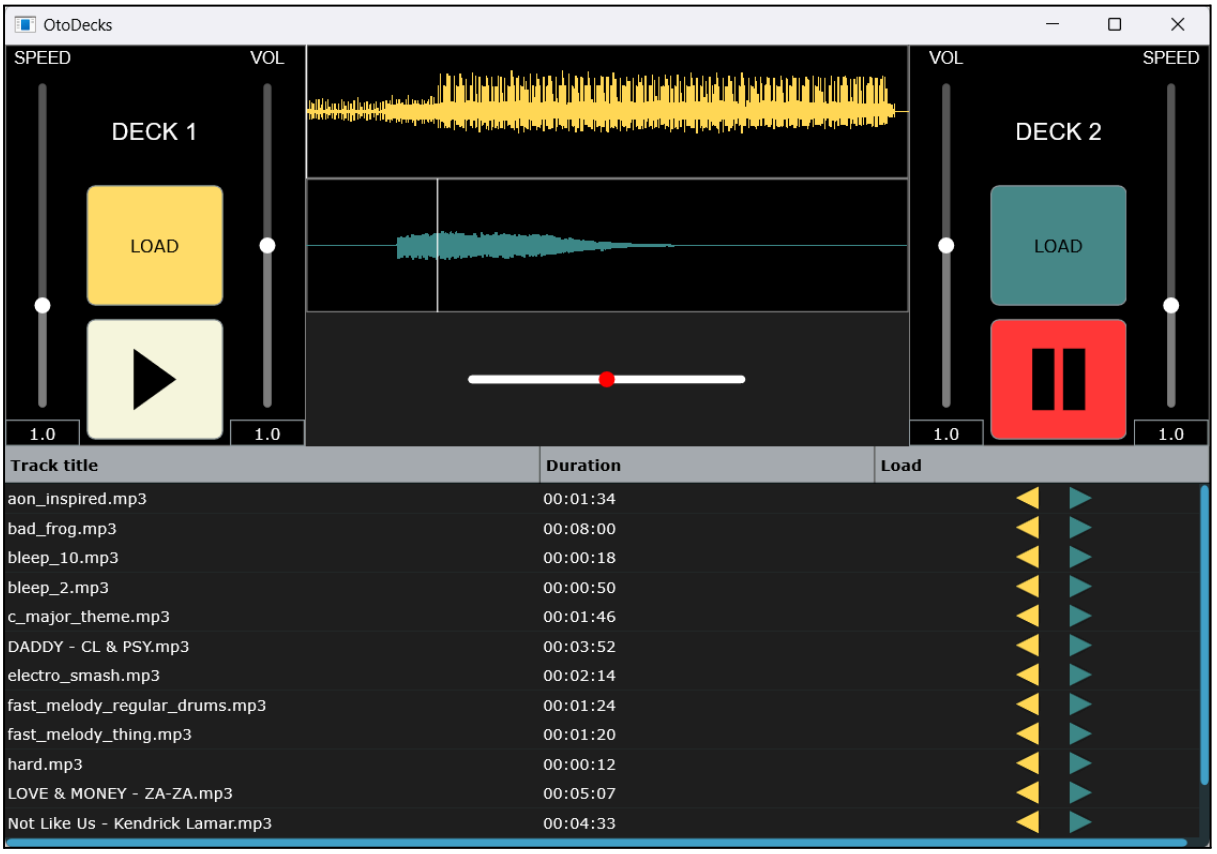
To allow users to control the playback speed, a `**Slider**` component (`**speedSlider**`) is used in the `**DeckGUI**` class. This slider is set to range from 0.1 to 3.0, with an initial value of 1.0 (normal speed). The slider displays values with one decimal place, and snapping is disabled for precise adjustments.

```
speedSlider.addListener(this);

== Sliders configurations ==========================================
/* Set range of sliders */
volSlider.setRange(0.0, 2.0);
speedSlider.setRange(0.1, 3.0);

/* Set default values for sliders */
volSlider.setValue(1.0);
speedSlider.setValue(1.0);

/* Limit slider values to increments of 0.1, display only 1 decimal place */
volSlider.setNumDecimalPlacesToDisplay(1);
speedSlider.setNumDecimalPlacesToDisplay(1);

/* Disable snapping to mouse position */
volSlider.setSliderSnapsToMousePosition(false);
speedSlider.setSliderSnapsToMousePosition(false);
```

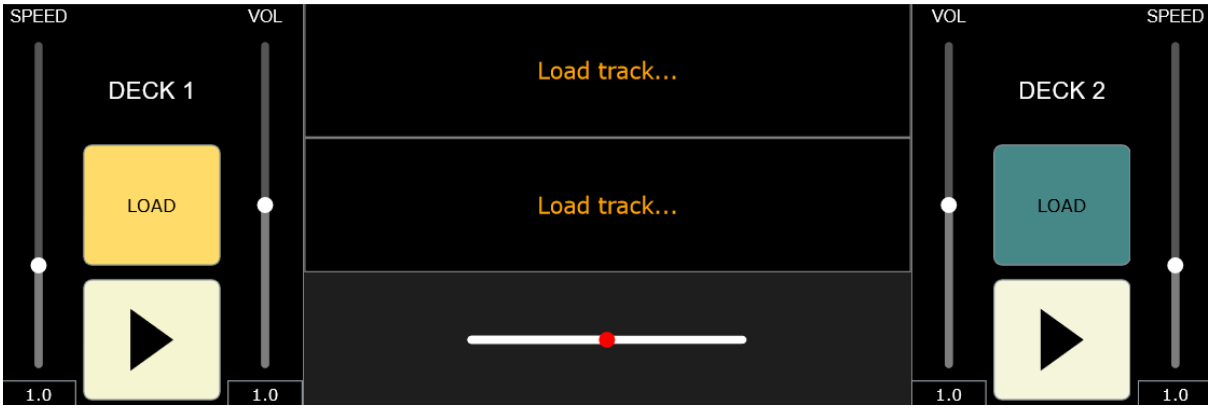
Whenever the slider value changes, the `**sliderValueChanged**()` method is invoked. If the slider in question is `**speedSlider**`, the value is rounded to one decimal place, and the `**setSpeed**()` method of the corresponding `**DJAudioPlayer**` (`player`) is called, updating the speed of the track.

```
void DeckGUI::sliderValueChanged(Slider* slider)
{
    if (slider == &volSlider)
    {
        double roundedValue = round(volSlider.getValue() * 10) / 10; // Round to 1 decimal place
        player->setGain(roundedValue);
    }
    if (slider == &speedSlider)
    {
        double roundedValue = round(speedSlider.getValue() * 10) / 10; // Round to 1 decimal place
        player->setSpeed(roundedValue);
    }
}
```

3. USER INTERFACE CUSTOMISATION



2.1. Dual Deck Symmetry Control GUI



To present a dual-deck layout, it was designed to have the ability to invert the layout for symmetry control. This is managed by the `**layoutInverted**` boolean variable of `**DeckGUI**` class, which toggles between two visual configurations.

```
public:
    DeckGUI(DJAudioPlayer* player,
            AudioFormatManager& formatManagerToUse,
            AudioThumbnailCache& cacheToUse,
            WaveformDisplay& waveformDisplay,
            bool layoutInverted = false);
    ~DeckGUI() override;
```

```
DeckGUI deckGUI1{ &player1, formatManager, thumbCache, waveformDisplay1, false };
DeckGUI deckGUI2{ &player2, formatManager, thumbCache, waveformDisplay2, true };
```

The layout is controlled within the `**resized()**` method, where the dimensions of the user interface components, such as the `**speedSlider**`, `**loadButton**`, `**playPauseButton**`, and `**volSlider**`, are adjusted according to whether the layout is inverted or not. The screen area is divided into four columns (`rowW`) and three rows (`rowH`), with a gap of 5 units between the elements to maintain spacing.

```
void DeckGUI::resized()
{
    double rowW = getWidth() / 4;
    double rowH = getHeight() / 3;
    double gap = 5;

    if (!layoutInverted)
    {
        speedSlider.setBounds(0, 4*gap, rowW, rowH * 3-4*gap);
        loadButton.setBounds(rowW+gap, rowH+gap, rowW*2-2*gap, rowH-2*gap);
        playPauseButton.setBounds(rowW+gap, rowH*2+gap, rowW*2-2*gap, rowH-2*gap);
        volSlider.setBounds(rowW*3, 4 * gap, rowW, rowH*3 - 4 * gap);
    }
    else
    {
        volSlider.setBounds(0, 4 * gap, rowW, rowH * 3 - 4* gap);
        loadButton.setBounds(rowW+gap, rowH+gap, rowW*2-2*gap, rowH-2*gap);
        playPauseButton.setBounds(rowW+gap, rowH*2+gap, rowW*2-2*gap, rowH-2*gap);
        speedSlider.setBounds(rowW * 3, 4 * gap, rowW, rowH * 3 - 4 * gap);
    }
}
```

If `**layoutInverted**` is set to `**false**`, the layout positions the **`speedSlider` on the left**, the `loadButton` and `playPauseButton` in the center, and the **`volSlider` on the far right**. Conversely, if `**layoutInverted**` is set to `**true**`, the **positions of the `speedSlider` and `volSlider` are swapped**, achieving a mirrored, inverted layout.

Furthermore, the **colors** of the `loadButton` are customized based on the layout orientation. When `layoutInverted` is `**false**`, the `loadButton` has a **yellowish color**, and when `layoutInverted` is `**true**`, the button's color changes to a **teal shade**.

```
if (!this->layoutInverted)
{
    /* deckGUI1 */
    g.drawText("DECK 1",
        loadButtonBounds.getX(),
        loadButtonBounds.getY() - 50, // Position text 50 pixels above the slider
        loadButtonBounds.getWidth(),
        20,
        Justification::centred,
        true);

    loadButton.setColour(TextButton::buttonColourId, Colour::fromRGB(255, 219, 88));
    loadButton.setColour(TextButton::textColourOnId, Colours::black);
    loadButton.setColour(TextButton::textColourOffId, Colours::black);
}
else
{
    /* deckGUI2 */
    g.drawText("DECK 2",
        loadButtonBounds.getX(),
        loadButtonBounds.getY() - 50, // Position text 50 pixels above the slider
        loadButtonBounds.getWidth(),
        20,
        Justification::centred,
        true);

    loadButton.setColour(TextButton::buttonColourId, Colour::fromRGB(62, 137, 137));
    loadButton.setColour(TextButton::textColourOnId, Colours::black);
    loadButton.setColour(TextButton::textColourOffId, Colours::black);
}
```

This also applied on the **waveformDisplay** of both players, so on `deckGUI1`, the load button and its waveformDisplay will be yellowish color and on `deckGUI2`, the load button and waveformDisplay will be teal shade.

```
public:
    WaveformDisplay(AudioFormatManager& formatManagerToUse,
                    AudioThumbnailCache& cacheToUse,
                    Colour waveformColour);
    ~WaveformDisplay() override;
```

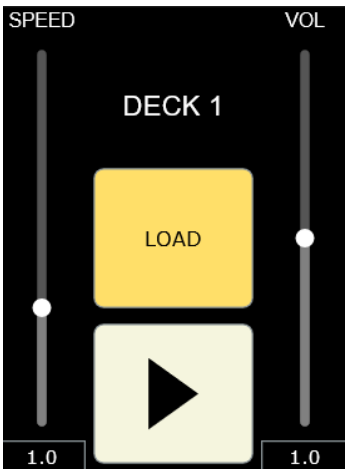
```
void WaveformDisplay::paint(juce::Graphics& g)
{
    g.fillAll(Colours::black);

    g.setColour(juce::Colours::grey);
    g.drawRect(getLocalBounds(), 1);   // draw an outline around the component

    g.setColour(juce::Colours::orange);
    if (fileLoaded && audioThumb.getNumChannels() > 0 && audioThumb.getTotalLength() > 0)
    {
        g.setColour(waveformColour);
        // Draw waveform
        audioThumb.drawChannel(g,
            getLocalBounds(),
            0,
            audioThumb.getTotalLength(),
            0,
            1.0f
        );
```

```
Colour deckGUI1_colour = Colour::fromRGB(255, 219, 88);
Colour deckGUI2_colour = Colour::fromRGB(62, 137, 137);
WaveformDisplay waveformDisplay1{ formatManager, thumbCache, deckGUI1_colour };
WaveformDisplay waveformDisplay2{ formatManager, thumbCache, deckGUI2_colour };
```

This provides visual feedback to the user, indicating the current state of the layout and enhancing the overall user experience by maintaining a balanced and symmetrical design.

2.2. Text in UI

To improve usability and clarity, **text labels** are added to sliders and deck components in the DJ application. This ensures that users can quickly identify what each slider controls and which deck they are interacting with.



2.2.1. Slider Labels

Text labels are displayed above the sliders to indicate their function. The speed slider is labeled `"**SPEED**"` and the volume slider is labeled `"**VOL**"`, improving the user's understanding of their controls. The text is rendered using a white color and an Arial font at 14pt size. The text is positioned 20 pixels above each slider for visibility.

```
g.setColour(Colours::white); // Set text for sliders
Font sliderFont = Font("Arial", 14.0f, Font::plain);
g.setFont(sliderFont);

Rectangle<int> speedSliderBounds = speedSlider.getBounds();
Rectangle<int> volSliderBounds = volSlider.getBounds();

g.drawText("SPEED",
    speedSliderBounds.getX(),
    speedSliderBounds.getY() - 20, // Position text 20 pixels above the slider
    speedSliderBounds.getWidth(),
    20,
    Justification::centred,
    true);

g.drawText("VOL",
    volSliderBounds.getX(),
    volSliderBounds.getY() - 20, // Position text 20 pixels above the slider
    volSliderBounds.getWidth(),
    20,
    Justification::centred,
    true);
```

2.2.2. Deck Identification

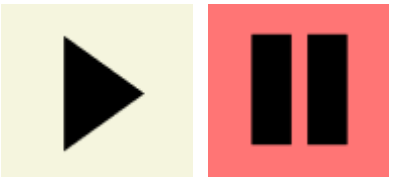
Each deck in the interface is labeled `**"DECK 1"**` or `**"DECK 2"**` depending on its position. If `layoutInverted` is `false`, the left deck is labeled `"DECK 1"`; otherwise, it is labeled `"DECK 2"`. The deck label is positioned 50 pixels above the `loadButton`, ensuring clear visibility without overlapping other components.

```
if (!this->layoutInverted)
{
    /* deckGUI1 */
    g.drawText("DECK 1",
        loadButtonBounds.getX(),
        loadButtonBounds.getY() - 50, // Position text 50 pixels above the slider
        loadButtonBounds.getWidth(),
        20,
        Justification::centred,
        true);
}
else
{
    /* deckGUI2 */
    g.drawText("DECK 2",
        loadButtonBounds.getX(),
        loadButtonBounds.getY() - 50, // Position text 50 pixels above the slider
        loadButtonBounds.getWidth(),
        20,
        Justification::centred,
        true);
}
```

These text elements enhance the overall user experience by making controls more intuitive and reducing confusion when adjusting settings or switching between decks.

2.3. Drawable `playPauseButton`

The `**playPauseButton**` is implemented as a `**DrawableButton**`, which allows it to display different icons for play and pause states instead of using a standard text button. This enhances the UI by making it visually intuitive.



2.3.1. Loading Play and Pause Icons

The icons for the play and pause states are stored as **SVG files** in the `**Assets**` directory. These files are loaded into `**std::unique_ptr<Drawable>**` objects using `**Drawable::createFromSVGFile()**`.

```
DrawableButton playPauseButton{ "playPauseButton", DrawableButton::ImageOnButtonBackground };
std::unique_ptr<Drawable> playIcon;
std::unique_ptr<Drawable> pauseIcon;
```

```
// Load the play and pause icons
File playSVGFile = File::getCurrentWorkingDirectory().getParentDirectory().getParentDirectory().getChildFile("Source/Assets/play.svg");
File pauseSVGFile = File::getCurrentWorkingDirectory().getParentDirectory().getParentDirectory().getChildFile("Source/Assets/pause.svg");
playIcon = Drawable::createFromSVGFile(playSVGFile);
pauseIcon = Drawable::createFromSVGFile(pauseSVGFile);
```

2.3.2. Setting Button Images

The `**playPauseButton**` dynamically switches between the play and pause icons depending on the player's state. When the player is not playing, the button displays the play icon, and when it is playing, it shows the pause icon. The `setImages()` function assigns these icons accordingly.

```
if (player->isPlaying == false)
{
    playPauseButton.setImages(playIcon.get(), playIcon.get(), playIcon.get(), playIcon.get(),
        pauseIcon.get(), pauseIcon.get(), pauseIcon.get(), pauseIcon.get());
}
else
{
    playPauseButton.setImages(pauseIcon.get(), pauseIcon.get(), pauseIcon.get(), pauseIcon.get(),
        playIcon.get(), playIcon.get(), playIcon.get(), playIcon.get());
}
```

2.3.3. Color Customization

The button is styled using different colors:
- When in the **play state**, the button has a beige (`RGB(245, 245, 220)`) background with black text.
- When in the **pause state**, it switches to a red (`RGB(255, 102, 102)`) background with white text.

```
// Play Button
playPauseButton.setColour(TextButton::buttonColourId, Colour::fromRGB(245, 245, 220));
playPauseButton.setColour(DrawableButton::textColourId, Colours::black);
// Pause Button
playPauseButton.setColour(TextButton::buttonOnColourId, Colour::fromRGB(255, 102, 102));
playPauseButton.setColour(DrawableButton::textColourOnId, Colours::white);
```

2.3.4. Toggling Play and Pause

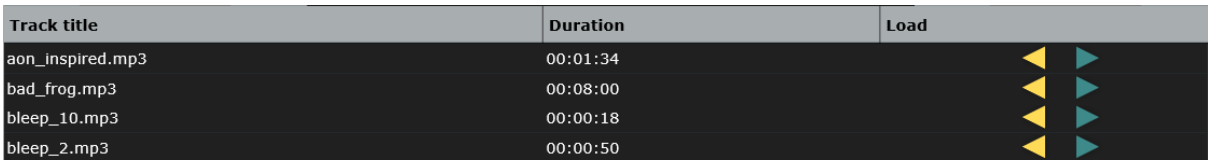
The button is configured with `**setClickingTogglesState(true)**`, ensuring that clicking it toggles between the play and pause states. The `**buttonClicked()**` function checks the `**isPlaying**` flag in the `**player**` and either starts or stops playback accordingly.

```
playPauseButton.setClickingTogglesState(true);
```

```
void DeckGUI::buttonClicked(Button* button)
{
    if (button == &playPauseButton)
    {
        if (player->isPlaying == false)
        {
            player->start();
        }
        else
        {
            player->stop();
        }
    }
```

This approach makes the play/pause control more intuitive, visually appealing, and user-friendly.

2.4. PlaylistComponent



The `**PlaylistComponent**` is responsible for displaying a structured list of tracks in a table format. It provides users with essential information about each track and options to load them into one of the two decks. The table consists of three columns: **Track title**, **Duration**, and **Load**, making it easy to manage and interact with the playlist.

```
tableComponent.getHeader().addColumn("Track title", 1, 400);
tableComponent.getHeader().addColumn("Duration", 2, 250);
tableComponent.getHeader().addColumn("Load", 3, 250);
```

The table header defines each column's purpose and width. The Track title column is 400 pixels wide and displays the name of the track. The Duration column, 250 pixels wide, shows the track's length in a `HH:MM:SS` format. The Load column, also 250 pixels wide, contains buttons that allow users to load a track into Deck 1 or Deck 2.

The `**paintCell**()` function handles the rendering of the table cells. It ensures that the track title and duration are properly displayed. Track titles appear in the first column and are left-aligned for easy readability. Similarly, the track duration is positioned in the second column and formatted to maintain a consistent appearance across all entries. White text is used for better visibility against the background.

```
void PlaylistComponent::paintCell(Graphics& g,
                                  int rowNumber,
                                  int columnId,
                                  int width,
                                  int height,
                                  bool rowIsSelected)
{
    g.setColour(Colours::white);
    if (columnId == 1) // Track titles
    {
        g.drawText(trackTitles[rowNumber],
            2, 0,
            width - 4, height,
            Justification::centredLeft,
            true);
    }
    if (columnId == 2) // Duration
    {
        g.drawText(trackDurations[rowNumber],
            2, 0,
            width - 4, height,
            Justification::centredLeft,
            true);
    }
}
```

The function `**formatDuration**()` is responsible for converting raw duration values from seconds into a standard `**HH:MM:SS**` format. It achieves this by extracting the number of hours, minutes, and seconds using integer division and formatting the output with leading zeros where necessary. This ensures that all track durations are displayed uniformly, regardless of their length.

```
std::string PlaylistComponent::formatDuration(double durationInSeconds)
{
    int totalSeconds = static_cast<int>(durationInSeconds); // Convert to integer
    int hours = totalSeconds / 3600;                        // Calculate hours
    int minutes = (totalSeconds % 3600) / 60;               // Calculate minutes
    int seconds = totalSeconds % 60;                        // Calculate seconds

    // Format into "HH:MM:SS"
    std::ostringstream stream;
    stream << std::setw(2) << std::setfill('0') << hours << ":"
        << std::setw(2) << std::setfill('0') << minutes << ":"
        << std::setw(2) << std::setfill('0') << seconds;

    return stream.str();
}
```

Plus as mentioned in *1.1.2. Load from PlaylistComponent*, each track row includes two **Load buttons** for user interaction. The `Load to Deck 1` button is colored yellow, while the `Load to Deck 2` button is teal. These buttons are dynamically created in `**refreshComponentForCell**()`, ensuring that they are properly positioned and assigned unique component IDs based on the row number. This allows tracks to be quickly and efficiently loaded into the appropriate deck for playback.

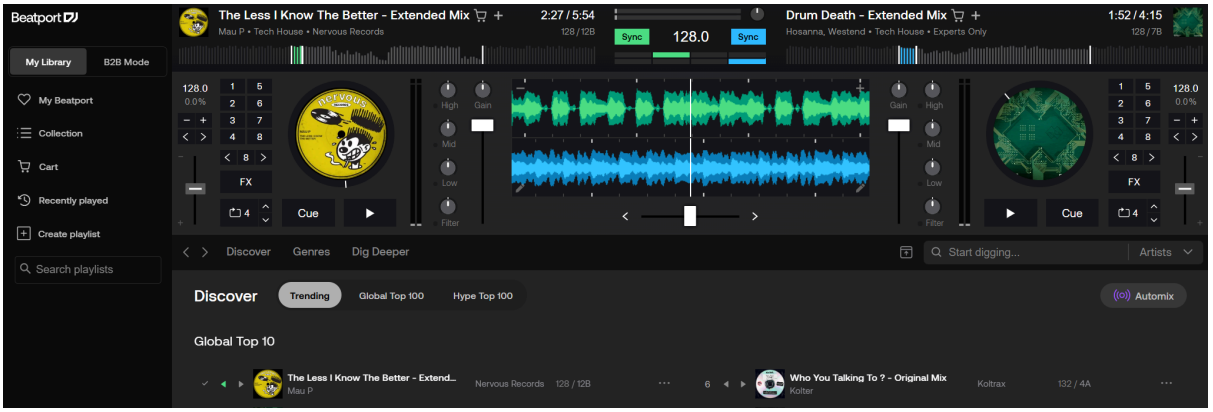
4. NEW FEATURE

3.1. Research

Two DJ applications were studied:

- Transitions DJ (*https://dj.app/*)

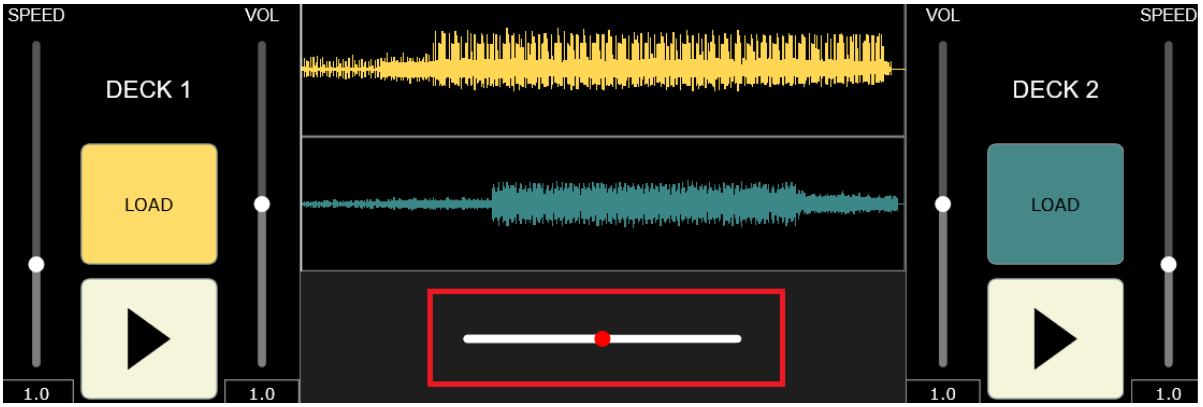
- Beatport DJ (*https://dj.beatport.com/*)



Both Transitions DJ and Beatport DJ have features for **controlling track loops**, allowing users to repeat specific sections of a song seamlessly. They also provide **EQ controls for high, mid, and low frequencies**, enabling precise audio adjustments. Additionally, both platforms feature a **slider for smooth track transitions**, helping DJs blend tracks effortlessly. Another key feature they offer is a **movable playhead**, which allows users to set the position of tracks manually for precise cueing and playback control.

3.2. Feature Identified to Implement

3.2.1. Transition Slider



The **Transition Slider** allows smooth audio blending between two tracks by adjusting their relative volumes.

```
Slider transitionSlider{ Slider::LinearHorizontal, Slider::NoTextBox };
```

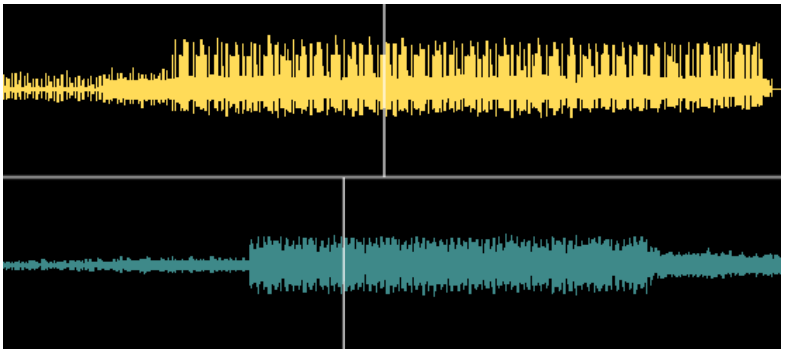
It is implemented using a JUCE Slider with a range from `0.0` to `2.0`, where `0.0` represents full volume for Player 1, `2.0` represents full volume for Player 2, and `1.0` is the balanced middle position. The slider is configured as Linear Horizontal with no text box, and its colors are set for better visibility.

```
// Configure the transition slider
transitionSlider.setRange(0.0, 2.0); // 0.0: player1 only, 2.0: player2 only
transitionSlider.setValue(1.0); // Start in the middle (both players at equal volume)
transitionSlider.addListener(this);
addAndMakeVisible(transitionSlider);
```

When the slider's value changes, the `sliderValueChanged` function calculates the gain for each player dynamically. The **gain formula** ensures that moving the slider left increases Player 1's volume while decreasing Player 2's volume, and vice versa. This is achieved by mapping the slider's value to a relative gain, where Player 1's gain decreases as the slider moves right, and Player 2's gain increases proportionally. The calculated gain values are then applied using `**setGain()**` for both players, creating a smooth transition effect between tracks.

```cpp
void MainComponent::sliderValueChanged(Slider* slider)
{
    if (slider == &transitionSlider)
    {
        // Get the transitionSlider value (ranges from 0.0 to 2.0)
        double sliderValue = transitionSlider.getValue();

        // From DeckGUI's volSlider setting
        double defaultVol = 1.0;
        double maxVol = 2.0;
        // Calculate the relative gain for player1 and player2 based on the slider value
        double player1Gain = defaultVol * (maxVol - sliderValue);   // slider moves left: p1 fade in, p2 fade out
        double player2Gain = defaultVol * sliderValue;              // slider moves right: p2 fade in, p1 fade out

        // Set the gain for both players
        player1.setGain(player1Gain);
        player2.setGain(player2Gain);
    }
}
```

3.2.2. Movable Playhead



The **Movable Playhead** feature allows users to click or drag on the waveform display to set the track's playback position dynamically. This is implemented using **mouse events** and a **callback mechanism** to update the track position in real-time.

In `**WaveformDisplay.h**`, the function `**setPositionRelative(double pos)**` updates the playhead's position based on a given relative value (0.0 to 1.0). A callback function `**setPlayheadMovedCallback(std::function<void(double)> callback)**` is defined to notify the **audio player** when the playhead is moved. The `**mouseDown**` and `**mouseDrag**` functions capture user interactions, calculating the new playhead position based on the **x-coordinate** of the mouse.

In `**WaveformDisplay.cpp**`, `**mouseDown**` and `**mouseDrag**` update the **playhead position** and trigger the callback function to reflect changes in the **track playback position**. The waveform and playhead are redrawn using `**repaint**()`, ensuring visual feedback when the user interacts with the waveform.

```
void WaveformDisplay::setPositionRelative(double pos)
{
    if (pos != position)
    {
        position = pos;
        repaint();
    }
}

void WaveformDisplay::setPlayheadMovedCallback(std::function<void(double)> callback)
{
    playheadMovedCallback = callback;
}

void WaveformDisplay::mouseDown(const juce::MouseEvent& event)
{
    // Calculate the relative position of the mouse click
    double newPosition = static_cast<double>(event.x) / getWidth();
    setPositionRelative(newPosition);

    // Trigger the callback to update playback position
    if (playheadMovedCallback)
    {
        playheadMovedCallback(newPosition);
    }
}

void WaveformDisplay::mouseDrag(const juce::MouseEvent& event)
{
    // Calculate the relative position of the mouse drag
    double newPosition = static_cast<double>(event.x) / getWidth();
    setPositionRelative(newPosition);

    // Trigger the callback to update playback position
    if (playheadMovedCallback)
    {
        playheadMovedCallback(newPosition);
    }
}
```

Finally, in `**DeckGUI.cpp**`, the `**setPlayheadMovedCallback**` function is linked to the audio player, allowing the track's playback position to be updated whenever the playhead is moved. This ensures seamless control over track navigation, enabling users to quickly jump to different sections of the song.

```
// When the playhead is moved, the callback is called with the new relative position.
waveformDisplay.setPlayheadMovedCallback([this](double newPosition) {
    player->setPositionRelative(newPosition);
    });
```

5. CONCLUSION

In conclusion, the development of Otodecks has enhanced its functionality and user experience, transforming it from a basic DJ application into a more dynamic tool. By customizing the user interface and incorporating a new feature inspired by industry-standard DJ applications, the project has expanded its capabilities, offering users a more intuitive and versatile platform. However, there is definitely still space for improvement, and it could be further developed in the future to meet evolving user needs and technical advancements.

6. REFERENCES

1. Transition DJ, *https://dj.app/*
2. Beatport DJ, *https://dj.beatport.com/*